

Transcription automatique audio guitare → MIDI

Soutenance pour le titre professionnel *Concepteur Développeur en Science des Données*

Damien DESSAUX

M2i

31 août 2026

1. Introduction
2. BC01 - Construction et alimentation d'une infrastructure de gestion de données
3. BC02 - Analyse exploratoire, descriptive et inférentielle de données
4. BC03 - Analyse prédictive de données structurées par l'intelligence artificielle
5. BC04 - Analyse prédictive de données non-structurées par l'intelligence artificielle
6. BC05 - Industrialisation d'un algorithme d'apprentissage automatique et automatisation des processus de décision
7. BC06 - Direction de projets de gestion de données
8. Conclusion

Introduction du projet

Contexte métier

La retranscription manuelle d'un audio de guitare vers un fichier MIDI est une tâche **chronophage**, nécessitant une **expertise musicale**. Une minute d'audio peut nécessiter **15 à 30 minutes de retranscription** selon la complexité du morceau.

Projet GuitarFlow

GuitarFlow souhaite développer un **Proof Of Concept** capable de transcrire automatique un audio de guitare en un fichier MIDI exploitable par un logiciel de MAO (Musique Assistée par Ordinateur). Le projet vise à réduire le temps de transcription, accélérer la création de supports pédagogiques ou encore faciliter les workflows de composition musicale.

Objectif du POC

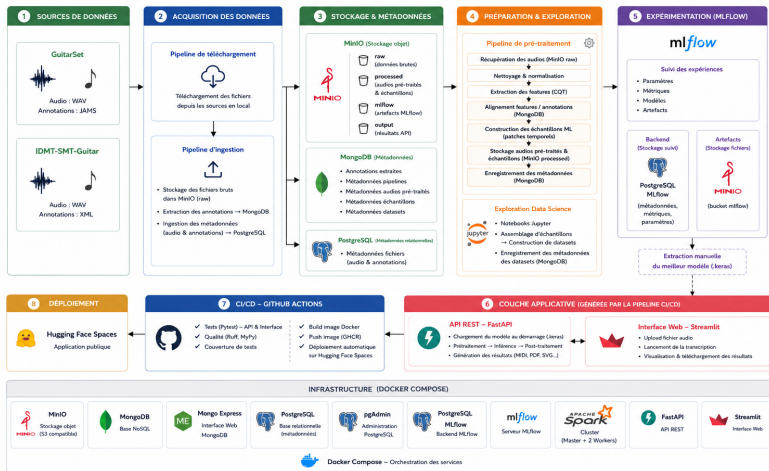
Valider la faisabilité technique d'une solution de Machine Learning de transformation **audio** → **MIDI**, avant une éventuelle phase d'industrialisation.

BC01 - Construction et alimentation d'une infrastructure de gestion de données

Infrastructure conceptualisée

ARCHITECTURE GLOBALE DE LA PLATEFORME

Guitarflow – Projet de transcription musicale



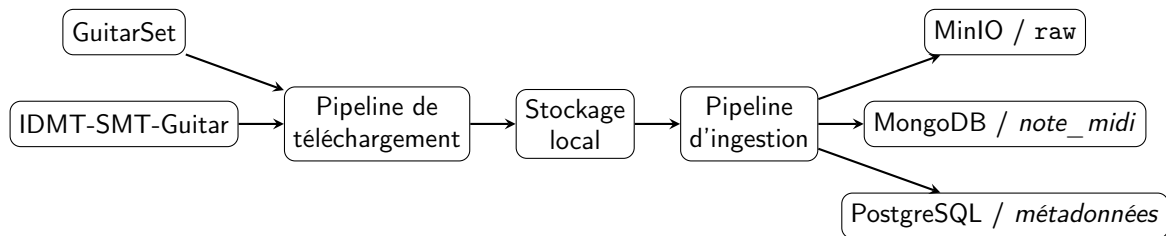
Composants principaux

- Data Lake : MinIO
- Bases de données : MongoDB & PostgreSQL
- Traitement : Python / Spark
- Expérimentation : MLflow
- Infrastructure : Docker-Compose

Choix d'architecture

Séparation des données, métadonnées et traitements afin de garantir traçabilité, évolutivité et reproductibilité.

Collecte et ingestion des données



Nature des données

- Fichiers audio (WAV).
- Fichiers d'annotation (JAMS et XML).
- Métadonnées associées.

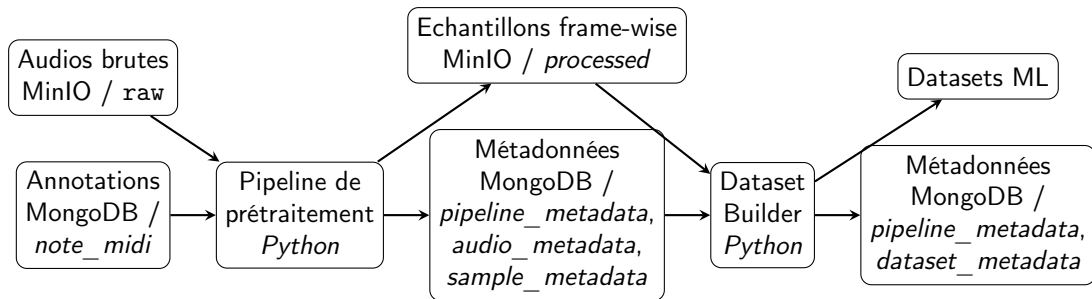
Collecte

- Téléchargement locale des archives (ZIP).
- Extraction locale des archives.

Ingestion

- Upload des datasets vers MinIO.
- Extraction des annotations MIDI dans MongoDB.
- Extraction des métadonnées dans PostgreSQL.

Organisation et transformation des données



Transformation

- Nettoyage du signal.
- Normalisation audio.
- Extraction des features.
- Alignement audio / annotations.

Production

- Fichiers audio normalisés.
- Echantillons frame-wise.
- Datasets de Machine Learning.

Préparation à la montée en charge

État actuel

- Volume compatible avec Python.
- Pipelines exécutées localement.
- Infrastructure Spark déjà provisionnée.

Évolution prévue

- Parallélisation des traitements.
- Augmentation du volume de données.
- Migration des pipelines vers Spark.

Choix d'architecture

Préparer la distribution des traitements en vue d'une phase d'industrialisation.

Reproductibilité de l'infrastructure

Reconstruction de l'infrastructure

```
$ cp .env.exemple .env  
$ docker-compose up -d
```

Reconstruction

- Services conteneurisés.
- Réseaux isolés.
- Volumes persistants.
- Configuration externalisée.

Traçabilité des données

- Données brutes conservées.
- URI centralisées dans PostgreSQL ou MongoDB.
- Traitements historisés.
- Datasets et expérimentations traçables.

Résultat

Une infrastructure **reconstructible** et prête à évoluer.

BC02 - Analyse exploratoire, descriptive et inférentielle de données

Présentation des datasets

Objectif de l'analyse

- Étudier la structure et la qualité des données audio de guitare
- Évaluer leur pertinence pour une tâche de transcription automatique vers MIDI

Datasets utilisés

- **GuitarSet [6]**
 - Enregistrements de guitare solo (audio + annotations)
 - Données alignées temporellement (value, time, duration)
 - Large couverture de styles de jeu
- **IDMT-SMT-Guitar [4]**
 - Dataset orienté reconnaissance musicale
 - Contient audio et annotations MIDI associées
 - Variété de conditions d'enregistrement

Enjeu métier

- Vérifier la compatibilité des données avec un pipeline de transcription audio → MIDI

Analyse du dataset GuitarSet

Objectif de l'analyse

- Comprendre la structure et la richesse du dataset
- Évaluer sa pertinence pour la transcription audio → MIDI

Axes d'analyse

- Structure des données (audio / annotations / alignement)
- Analyse musicale descriptive (pitch, densité, accords)
- Analyse du signal audio (spectrogrammes, dynamique)
- Qualité des annotations temporelles

Conclusion intermédiaire

- Dataset riche mais partiellement contraint
- Bonne base pour apprentissage supervisé
- Complément nécessaire avec IDMT-SMT-Guitar

Analyse du dataset IDMT-SMT-Guitar

Analyse d'un dataset Frame-Wise

BC03 - Analyse prédictive de données structurées par l'intelligence artificielle

BC04 - Analyse prédictive de données non-structurées par l'intelligence artificielle

Architecture du modèle

Choix des hyperparamètres

Pipeline d'entraînement

BC05 - Industrialisation d'un algorithme d'apprentissage automatique et automatisisation des processus de décision

BC06 - Direction de projets de gestion de données

Présentation du projet

GuitarFlow est une entreprise développant un prototype de service d'intelligence artificielle destiné à la transcription automatique d'enregistrements audio de guitare en fichiers MIDI exploitables en MAO (Musique Assistée par Ordinateur).

Constat métier

La retranscription manuelle d'une performance de guitare vers un format éditable constitue une tâche **chronophage**, nécessitant une **expertise musicale avancée** et difficilement **industrialisation-compatible** pour des usages pédagogiques ou créatifs.

Opportunité

Le projet vise à :

- réduire le temps de transcription audio → MIDI,
- faciliter la production de contenus pédagogiques,
- accélérer les workflows de composition musicale,
- proposer un démonstrateur d'IA appliquée au signal audio.

Objectif principal

Développer un prototype de service capable de :

- recevoir un fichier audio guitare,
- générer automatiquement un fichier MIDI exploitable,
- produire une visualisation de type piano-roll,
- exposer le modèle via une API web déployée sur Hugging Face.

Fonctionnalités incluses

- Audio guitare mono instrument
- Guitare acoustique et électrique
- Accordage standard (EADGBE)
- Génération MIDI
- Visualisation piano-roll
- API REST d'inférence
- Conteneurisation Docker

Fonctionnalités hors périmètre

- Multi-instruments
- Accordages alternatifs
- Temps réel
- Techniques de jeu avancées
- Application mobile native
- Architecture cloud managée

Analyse des besoins et exigences

Exigences fonctionnelles

- Accepter un enregistrement audio de guitare au format WAV.
- Produire automatiquement une transcription au format MIDI exploitable dans un logiciel de MAO.
- Permettre la visualisation des notes détectées sous forme de piano-roll.
- Exposer le service via une API REST et une interface web.
- Garantir la prise en charge d'enregistrements de guitare acoustique et électrique.
- Permettre l'évaluation et la comparaison de plusieurs approches de transcription.

Exigences non fonctionnelles

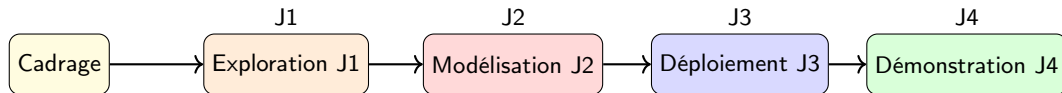
- Temps d'inférence inférieur à 10 secondes pour 1 minute d'audio.
- Reproductibilité complète des expérimentations et des résultats.
- Traçabilité des jeux de données, modèles et métriques.
- Déploiement reproductible par conteneurisation Docker.
- Maintenabilité assurée par une architecture modulaire, des tests automatisés et une documentation technique.

Indicateur	Objectif cible
F1-score de transcription polyphonique	$\geq 90 \%$
Temps moyen d'inférence	< 10 s pour 1 min d'audio
Disponibilité du prototype	Démonstration fonctionnelle avant le 31/07/2026
Déploiement	API accessible via une interface web
Taux de correction manuelle	$< 10 \%$

Approche de pilotage

Le projet a été conduit selon une démarche incrémentale articulée autour de quatre jalons de validation permettant de sécuriser progressivement les différentes phases du POC.

Jalon	Semaine	Critère de validation
J1 – Go Datasets	S4	Données validées et exploitables
J2 – Go Modèle	S6	Baseline opérationnelle
J3 – Go Prototype	S8	Pipeline complet exécutable
J4 – Go Démonstration	S10	API et interface accessibles



► Voir le Gantt détaillé

Risque	Impact	Prob.	Mitigation
Restrictions de licence sur les data-sets	Élevé	Moyen	Validation juridique des sources avant intégration
Volume de données insuffisant	Élevé	Élevé	Data augmentation + transfer learning
Performances insuffisantes en polyphonie	Élevé	Moyen	Benchmark de plusieurs architectures (baseline / CNN / CRNN)
Temps d'inférence trop élevé	Moyen	Moyen	Optimisation du modèle + quantization
Variabilité de qualité audio	Moyen	Élevé	Normalisation et standardisation du signal audio

Contraintes budgétaires et choix d'architecture

Contrainte principale

Le projet s'inscrit dans une logique de prototypage à faible coût, orientée preuve de concept.

Principes d'architecture retenus

- Priorité aux solutions open source
- Exécution locale ou infrastructure légère
- Limitation des coûts de calcul et d'entraînement
- Absence d'infrastructure cloud industrielle en phase 1

Conséquences techniques

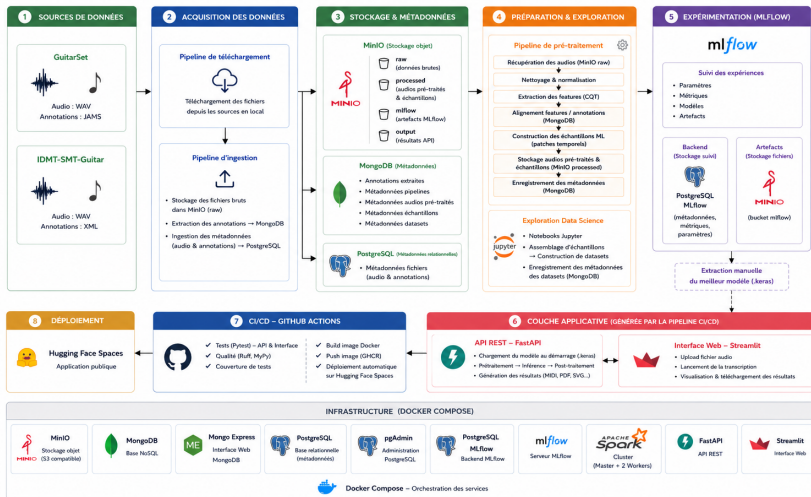
- Utilisation de Docker Compose pour orchestration locale
- Stockage local / MinIO plutôt que cloud managé
- Entraînement optimisé sur jeux de données limités
- Choix de modèles compatibles CPU

Contraintes réglementaires et licences

- GuitarSet : licence compatible Creative Commons Attribution 4.0 International
- IDMT-SMT-Guitar : licence non commerciale limitant l'usage à un POC

ARCHITECTURE GLOBALE DE LA PLATEFORME

Guitarflow – Projet de transcription musicale



Réalisation du POC

Approche de réalisation

Le POC a été construit selon une logique de pipeline end-to-end couvrant l'ensemble du cycle de vie de la donnée : ingestion, traitement, entraînement et mise à disposition du modèle.

Livrables techniques

- Pipelined de téléchargement et d'ingestion des données
- Pipeline de prétraitement audio et extraction de caractéristiques
- Pipeline de construction du dataset frame-wise
- Pipeline d'entraînement et d'évaluation des modèles
- Modèle entraîné et évalué pour la transcription audio → MIDI
- API REST d'inférence (FastAPI)
- Conteneurisation du système via Docker / Docker Compose
- Interface de démonstration déployée sur Hugging Face Spaces

Livrables documentaires

- Note de cadrage et cahier des charges
- Documentation technique des pipelines et de l'architecture
- Guide d'installation et de déploiement

Performance du système

Indicateur	Résultat
CV F1-score transcription polyphonique	78 ($\pm$ 3) %
Temps d'inférence moyen	? s pour 1 min d'audio
Génération de fichier MIDI	Non
Visualisation piano-roll	Non
API d'inférence fonctionnelle	Non
Interface de démonstration déployée	Non
Taux de correction manuelle	? %

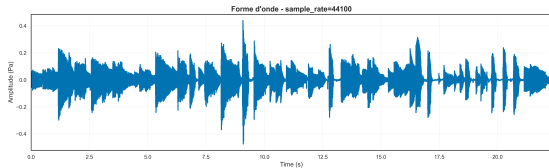
Synthèse

Le POC ne démontre pas encore la faisabilité technique et la valeur métier de la transcription audio → MIDI.

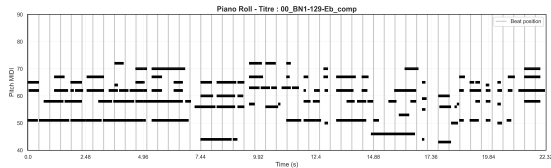
Exemple de transcription audio vers MIDI

Transformation réalisée par le système

Audio (.wav) → Modèle de transcription → MIDI (.mid)



Forme d'onde de l'enregistrement guitare



Piano-roll généré à partir de la transcription MIDI

Synthèse

Le système transforme automatiquement un signal audio brut en représentation symbolique exploitable par les logiciels de MAO.

Gouvernance et qualité

Gouvernance des données

- Datasets stockés en brut dans MinIO.
- PostgreSQL utilisé comme **source de vérité**.

Traçabilité des traitements

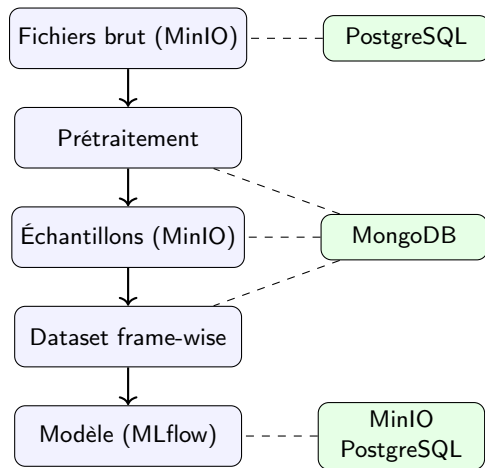
- Métadonnées stockées dans MongoDB.
- Versionnement des prétraitements appliqués.
- Versionnement des datasets générés.

Qualité logicielle

- GitHub / Docker / Ruff + Mypy / Tests / Mlflow

Résultat

Chaque modèle peut être relié aux données sources, aux prétraitements appliqués et aux dataset frame-wise sur lequel il s'est entraîné.



Traçabilité complète des données, traitements et modèles.

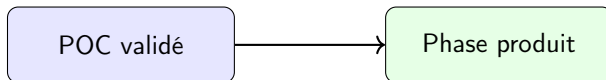
Conclusion

Bilan du projet

- Faisabilité technique démontrée
- Pipeline end-to-end opérationnel
- Génération automatique de fichiers MIDI
- API REST et interface web déployées
- Traçabilité et reproductibilité assurées

Perspectives

- Amélioration des performances en polyphonie complexe
- Optimisation des temps d'inférence
- Déploiement d'une API dédiée en production
- Détection avancée des techniques de jeu
- Support multi-instruments



Conclusion

Le projet valide la faisabilité d'une solution de transcription audio vers MIDI et constitue une base solide pour une phase d'industrialisation.

Je vous remercie pour votre attention.
Avez-vous des questions ?

9. Diagramme de Gantt détaillé

10. Stack technologique du projet

11. Bibliographie

Diagramme de Gantt détaillé

Phase	Tâches	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Cadrage	Validation note de cadrage										
Exploration	Ingestion des données										
	Analyse exploratoire des données										
	Preprocessing audio et extraction des features										
	Construction des datasets frame-wise				J1						
Modélisation	Benchmark des modèles candidats						J2				
	Implémentation du pipeline ML complet								J3		
Déploiement	Développement de l'API d'inférence										
	Dockerisation et déploiement sur Hugging Face Spaces										
Démonstration	Présentation phase1										J4

Stack technologique du projet

Groupe	Thème	Technologie (version)
Data Engineering	Stockage objet Base relationnelle Base document	MinIO (latest) PostgreSQL (latest) MongoDB (latest)
Traitement des données	Calcul distribué Data manipulation Calcul scientifique Audio processing Annotation audio MIDI processing	Apache Spark (latest) Pandas (latest) NumPy (latest) Librosa (latest) JAMS (latest) pretty_midi (latest)
Machine Learning	Machine Learning Deep Learning Tracking ML	Scikit-Learn (latest) TensorFlow (latest) MLflow (latest)
API & Déploiement	API REST Schéma data Conteneurisation Interface web	FastAPI (latest) Pydantic (latest) Docker (latest) Streamlit (latest)



Aurélien Géron.

Machine Learning avec Scikit-Learn : mise en oeuvre et cas concrets.

Dunod, Paris, 3 edition, 2023.



Aurélien Géron.

Deep Learning avec Keras et TensorFlow : mise en oeuvre et cas concrets.

Dunod, Paris, 3 edition, 2024.



Christian Kehling, Jakob Abeßer, Christian Dittmar, and Gerald Schuller.

Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters.

In *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx)*, Erlangen, Germany, 2014.



Christian Kehling, Andreas Männchen, and Arndt Eppler.

Idmt-smt-guitar dataset, 2023.



Gilbert Saporta.

Probabilités, analyse des données et statistique.

Technip, Paris, 3 edition, 2011.



Qiaoxi Xi, Seth Dittmer, Justin Pauwels, and Mark Sandler.

Guitarset : A dataset for guitar transcription.

In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, Paris, France, 2018.